

## **Department of Computer and Software Engineering-ITU**

### **CE203L: Computer Organization and Architecture Lab**

|                                          |                                 |
|------------------------------------------|---------------------------------|
| <b>Course Instructor: Mr. Aftab Alam</b> | <b>Dated:</b>                   |
| <b>Lab Engineer: Muhammad Usama Riaz</b> | <b>Semester: 4<sup>th</sup></b> |
| <b>Session: 2024-2028</b>                | <b>Batch: BSCE24</b>            |

## **Lab 12. Implementing Top Module and Test Bench for RISC-V Processor**

| <b>Name</b> | <b>Roll No</b> | <b>Obtained Marks/35</b> |
|-------------|----------------|--------------------------|
|             |                |                          |

Checked on: \_\_\_\_\_

Signature: \_\_\_\_\_

## Objective

The goal of this handout is to design and implement the top module for MIPS processor and verify its functionality.

## Equipment and Component

| Component Description  | Value     | Quantity |
|------------------------|-----------|----------|
| Computer/Xilinx Vivado | On Campus | 1        |

## Conduct of Lab

1. Students are required to perform this experiment individually.
2. In case the lab experiment is not understood, the students are advised to seek help from the course instructor, lab engineers, assigned teaching assistants (TA) and lab attendants.

## Theory and Background

So far we have only viewed the signals from top module. To view the signals from its child modules, we will have to call them in the top modules test bench.

We use following Steps to add the signals in the Test Bench

-First create the wire of the test signal which you want to add in the test bench.

-After the instantiation, use the following syntax to assign the value of a child module.

assign Signal=<**Instance** name of TOP module> . < **instance** name of child module> . < Name of the signal to call>

If the signal is within another module which is within the child i.e., child of child module then use the same syntax and add another instance name at appropriate place.

Here is an example, to see B in test Bench we use

Test Bench

```
TOP uut(xxx,xxx,xxx)
```

```
module TOP
```

```
M1 mod1(xxx,xxx,xxx)
```

```
M2 mod2(.A(x),.B(x));
```

```
module M2(A,B) Input A;
```

```
output B; assign B=A;
```

```
assign data=uut.mod1.mod2.B;
```

Remember that we use **instance names**, not **modules names** when adding a signal in test bench

Create a wire of the new signal before top module **instantiation**, in the test bench

Assign that wire the value right after the instantiation line and before the initial block, according to the specified syntax.

We will need following signals in our test bench. To see the different stages of instruction

PC\_Out

Instr

RF\_Data1

RF\_Data2

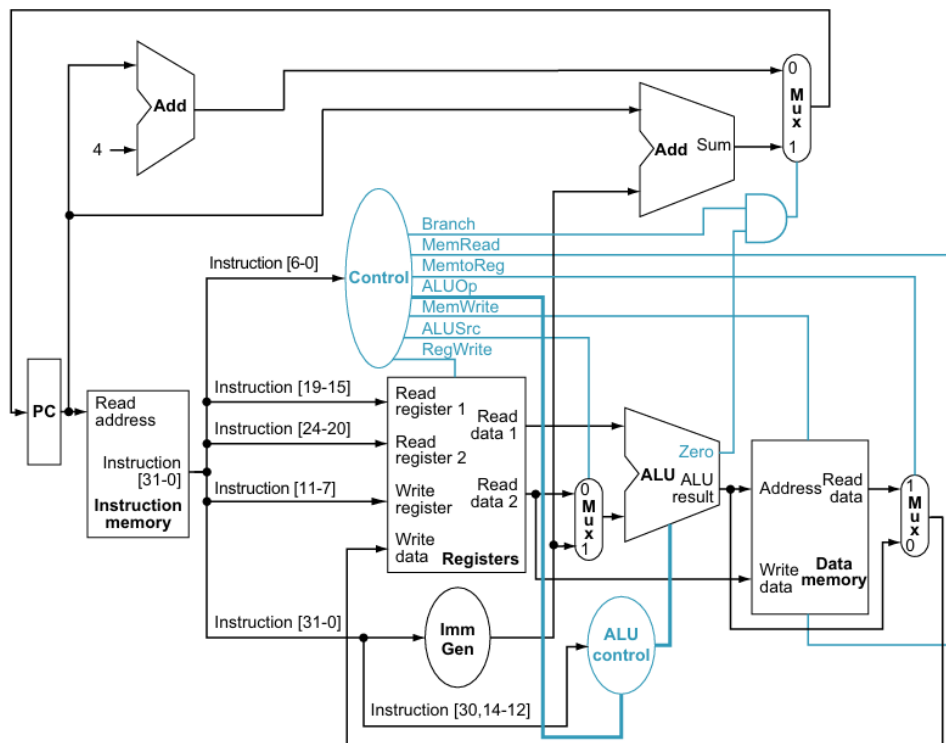
ALU\_Out

end

## Lab Tasks

### Task:

As you have implemented the RISC-V datapath and control unit in previous labs, now implement the top module and test bench for your RISC-V processor.



# RISC-V Single Cycle Processor



## Assessment Rubric for Lab

| Performance metric                          | CLO | Able to complete the task over 80% (4-5)                                                                                                          | Able to complete the task 50-80% (2-3)                                                                                 | Able to complete the task below 50% (0-1)                                                               | Marks |
|---------------------------------------------|-----|---------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|-------|
| 1. Realization of experiment                | 1   | Implements the Verilog code correctly, executes without errors, well-structured module definitions, proper signal naming, and complete testbench. | Implements the Verilog code with minor errors, partial testbench coverage, and minimal use of meaningful signal names. | Code does not execute due to syntax errors, missing modules, or incorrect logic. No testbench included. |       |
| 2. Conducting experiment                    | 1   | Able to modify Verilog code as required, answers all conceptual and implementation questions correctly.                                           | Partially able to modify code, some incorrect or incomplete answers.                                                   | Unable to modify the code, incorrect or no answers to the questions.                                    |       |
| 3. Computer use                             | 2   | Submits lab reports and code on time with appropriate documentation and organization.                                                             | Submits lab reports and code with minor delays, lacks proper documentation.                                            | Lab report or code submission is missing or significantly late.                                         |       |
| 4. Teamwork                                 | 3   | Actively engages in discussions, collaborates effectively, and contributes significantly to group work.                                           | Collaborates but with minimal engagement, contribution could be improved.                                              | Distracts or discourages team members, minimal or no contribution.                                      |       |
| 5. Laboratory safety and disciplinary rules | 3   | Verilog code is well-commented, explaining each module and functionality clearly.                                                                 | Verilog code has comments, but they are not informative or insufficient.                                               | No comments in the code, making it difficult to understand.                                             |       |
| 6. Data collection                          | 3   | Waveforms and simulation results are well-organized, properly labeled, and demonstrate correct functionality.                                     | Waveforms and simulation results are partially organized but lack clarity or some required tests.                      | Poorly captured waveforms, incorrect or missing simulation results.                                     |       |
| 7. Data analysis                            | 4   | Efficient and optimized Verilog design, easy to understand and scalable.                                                                          | Functional design but lacks optimization or clarity in certain areas.                                                  | Inefficient or difficult-to-maintain design, hard to interpret logic.                                   |       |
| <b>Total (out of 35):</b>                   |     |                                                                                                                                                   |                                                                                                                        |                                                                                                         |       |